后训练学习计划 • 第 9 周（Phase 2 • RLHF / RLVR 与 GRPO：跑通第一个强化学习循环，里程碑 M3，11/2-11/8）

本周定位：**把「优化一个比较损失」升级为「在环境里采样、被奖励打分」**。PPO 只到能白板讲清五角色与 KL/GAE，**不实操**；GRPO 要真跑——组内基线手推到能默写，规则奖励验证器自己写，在 GSM8K 子集上画出 reward / 输出长度 / KL 三条曲线。产出里程碑 M3 与第 3 篇博客《GRPO 第一次实操：reward 曲线教会我的事》。

用法说明：每天先看「今日总览」，再从任务 1 逐条做；每个任务都有 **完成判据**，判据不满足不要进入下一个任务。**加粗 = 硬性要求**，灰色小字块 = 注释（可跳过，不影响主线）。

一、本周总览

天	主题	核心硬产出	预计
Day 1 周一	RL 视角复盘 + PPO 五角色与 KL/GAE 白板	五角色图 + GAE 递推默写	6h
Day 2 周二	GRPO 组内基线推导 + RLVR + 与 PPO 对比表	推导 5 步 + 7 维对比表 + 闭卷默写	6h
Day 3 周三	规则奖励验证器三件套 + 单测	提取器/格式校验/CEFR 分类 单测全绿	6h
Day 4 周四	verl 环境搭建（**本周最大风险**）+ 降级路线打通	至少一条能跑出 reward 的路径	7h
Day 5 周五	GSM8K 子集跑 GRPO + 三条曲线	reward/长度/KL 曲线 + 健康判据结论	6h
Day 6 周六	准确率对照 + hacking 小实验 + 失败案例 + 博客	前后准确率表 + 3 条失败案例 + 博客发布	7h
Day 7 周日	补漏 + 闭卷复测 + 总验收 + 下周预习	验收 ≥6/7 项通过 = M3 达成	4h

注：**本周卡时上限 16h，预算目标 ≈¥35**。主力 4090 24G：Day 4 约 3h、Day 5 约 5h、Day 6 约 3h，合计 11h（¥16-28），留 3h 重跑余量。**A100 80G 短租 2h 是「额外许可项」**（¥6-12/时）：只在 Day 5 rollout 反复爆显存且降级路线已跑通时才动用，**一旦动用必须同步把 Day 6 任务 2 的 hacking 实验砍成本地 0.5B 小样本**（省 3h 4090），否则本周必超 ¥35。API 另计 ≤¥8（Day 6 判卷一致性抽查），全周上限 ¥43。

注：**本周最大风险是 Day 4 的 verl 环境**：依赖冲突（vLLM / torch / CUDA 三者版本必须同时对齐）、rollout 引擎显存、多版本 API 漂移。**降级路线是主线的一部分，不是备选**：verl 装不动 → 立刻切 TRL `GRPOTrainer`（0.5B + 小样本，单卡甚至本地就能跑通）。GRPO 的**原理**与**能不能跑通一个 RL 循环**这两件事，降级路线一件都不少。

注：**明确写在最前面：跑不收敛也算本周完成**（总计划第 6 节风险预案第 1 条）。判据看的是「三条曲线有没有画出来 + 有没有一句带数字的诊断」，不是「准确率涨了多少」。失败曲线与诊断过程本身就是面试素材——第 14 周的「失败案例复盘」清单，本周就是它最大的一批原料。**但「不收敛」必须是被记录、被分析的不收敛**：只有一张看不出所以然的图，不算完成。

注：本周不做 tutor、不碰 M4。周 10 才开工，提前动手会把周 10 的立项文档写成事后总结。

二、开始前：环境 / 数据 / 账号（约 45 分钟，周日晚上做完）

1. **本地 venv（沿用上周那个，别开第三个环境）**：

```
cd E:/study/posttrain && source .venv-dpo/Scripts/activate
pip show torch | grep '^Version' > torch-pin.txt          # 先记下现在的 torch 版本，见注
pip install -U "trl>=0.11" datasets sympy matplotlib numpy
python -c "import torch, trl, transformers; print(torch.__version__, trl.__version__,
transformers.__version__)"
```

> 注：**别在本机装 `flash-attn`**——它在 Windows 上基本编译不过，而且没有它 GRPO 0.5B 冒烟照样跑（PyTorch 自带 SDPA）。搜「windows 编译 flash attention」是两小时的坑，本周不付这个代价。同理 `lm-eval` 今天也不装：GSM8K 的评测我们自己写 `eval_acc.py`（Day 6），少一个会顶掉 torch 版本的依赖。
>
> 注：**Windows + conda 的 `OMP: Error #15` 上周已经处理过**（独立 venv 就是正解，临时绕过用 `export KMP_DUPLICATE_LIB_OK=TRUE`）。这周多一个触发点：任何后续 pip 安装都可能把 torch 顶掉版本，于是 `import torch` 又开始报错或 CUDA 突然不可用。**所以上面先落一份 `torch-pin.txt`**：版本变了就 `pip install torch==<pin 里的版本>` 装回去，别在 conda base 里修，越修越乱。

2. 建目录，并把「Day 4 开卡第一件事」写在显眼处：

```
mkdir -p E:/study/posttrain/week09/{day1,day2,day3,day4,day5,day6,day7}
# Day 4 开机后立刻把 nvidia-smi + torch/vllm 版本存进 day4/instance-versions.txt（明天所有报错靠它定位）
```

3. 上周 gaps 清账：从 `week08/gaps.md` 取这两条（其余不动）：①「长度偏置只观察到、没修」→ 本周 Day 6 任务 2 正面制造并诊断；②「held-out 与训练集 accuracy 差异的解释」→ 本周 Day 2 任务 1 推导 GRPO 时，组内优势归一化正是同一件事的另一面。
4. AutoDL 余额确认 ≥¥60（本周 GPU ≤¥35 + 下周 M4 需要 ¥30）；模型走 `export HF_ENDPOINT=https://hf-mirror.com` 或魔搭直下 `modelscope download --model Qwen/Qwen2.5-0.5B --local_dir ./qwen05b`。

Day 1 • 周一：PPO 五角色与 KL / GAE —— 只到能白板讲清

今日总览（6h）：今天不开卡、不训练。目标是**能在白板上把 PPO 的五个角色、KL 惩罚的位置、GAE 的递推讲清楚**，并且能回答「GRPO 去掉了哪一个、为什么去得掉」。**明确不实操 PPO**——四个网络同时在线、还要配 rollout 引擎，个人预算与卡时都撑不住，讲不清才是真问题。

任务 1：把前八周用 RL 的语言重画一遍（约 60 分钟）

在 `week09/day1/rl-map.md` 里填这张表（先默写后对照），它是今天所有内容的坐标系：

阶段	谁是策略 π	谁是环境	奖励从哪来	需要在线采样吗	需要几个模型
预训练 / M1 小 GPT	模型	语料本身	下一 token 的 CE（隐式）	否	1
SFT（周 6）	模型	指令数据集	只在 assistant span 上的 CE	否	1
DPO（周 8）	模型	偏好数据集（静态）	隐式奖励 = β·log(π/π_ref)	否	2（policy+ref）

阶段	谁是策略 π	谁是环境	奖励从哪来	需要在线采样吗	需要几个模型
RLHF-PPO	模型	**模型自己的输出分布**	RM 打分	**是（rollout）**	**5 个角色**
RLVR-GRPO（本周）	模型	模型自己的输出 + 验证器	**规则验证器**（对/错/格式）	**是**	policy + ref + 采样引擎

完成判据：表格五列全部填满（**不许写「类似 DPO」这类含混话**）；表下写一行「DPO 与 PPO 最大的分水岭」，答案必须点出**在线采样**与**分布外**这两个词。

> 注：分水岭一句话讲透：DPO 是在**别人采好的答案**上学比较，模型不会遇到「自己生出来的东西」；PPO/GRPO 是在**模型自己现在会生成的东西**上学，因此它既能超出数据（这是效果上限的来源），也会遇到验证器没见过、或模型能骗过去的输出（这是 hacking 与不收敛的来源）。面试官问「为什么 RL 比 DPO 贵且不稳」，你要能从这句话展开，而不是回答「因为 RL 难调」。

任务 2：PPO 目标函数逐项 + 五角色（约 120 分钟）

1. 逐项读懂这两式，每一部分标出「谁提供、什么时候算、梯度会不会流过它」：

```
L_PPO = E_t [ min( r_t(θ)·Â_t , clip(r_t(θ), 1-ε, 1+ε)·Â_t ) ]  -  β · KL( π_θ(·|x) ‖ π_ref(·|x) )
r_t(θ) = π_θ(a_t | s_t) / π_θ_old(a_t | s_t)        # 概率比：新策略 vs 采样本批数据时的旧策略
Â_t    = GAE(value 估计, reward, γ, λ)              # 由 Critic 给出的优势
```

```
- 为什么要 **clip** 而不是只用 `r_t·Â`：概率比一旦跑飞，一个高优势样本能把参数拖到灾难性更新；clip 把单步更新幅度限制住，这是 PPO 比普通 policy gradient 稳的核心。
- `θ_old` 是**采样时刻的策略快照**，不流梯度；`π_ref` 是 **SFT 权重**，冻结、不流梯度。这两点分开记，混了就讲不清。
```

2. 五个角色写成一张「显存账单」表（面试最爱追的「PPO 为什么贵」）：

角色	干什么	是否训练	是否要存梯度/优化器态
policy（actor）	生成回答、被更新	是	是
value（critic）	估每步价值，给 GAE 用	是	是
reference	锚定 SFT，算 KL	否	只存权重
reward model	给整条回答打分	否（离线）	只存权重
rollout 引擎	高效采样（vLLM 之类）	—	自己还要占一块显存

3. 用 7B 粗算一次量级并写进 NOTES.md：4 份网络各 ≈14GB（bf16 两字节/参），policy 与 value 还各有梯度 ≈14GB + AdamW 态 ≈56GB（8 字节/参）——**手算结论落在「远超 8 张 4090」**，这就是为什么个人实验必须走「去掉 critic 的 GRPO」而不是 PPO。**写清这是按公式估的量级，不是你实测的数字**。

完成判据：目标函数里四个符号（`r_t`、`clip`、`Â_t`、KL 项）各有 ≥2 行解释；五角色表五行「是否训练 / 是否存优化器态」两列无空格；NOTES.md 有那段 7B 显存量级手算（含「这是估算」四个字）。

> 注：Critic 与 RM 的区别是被追问率极高的一题，标准答案两条：① **粒度**：RM 给整条回答一个标量（句级），Critic 给每个 token 位置一个价值（token 级）；② **用途**：RM 是「外部裁判」，Critic 是「内部基线」，存在的唯一理由是把 advantage 的方差压下来。能答出「Critic 不产生奖励、只负责把奖励拆成每步的贡献」，这题就过了。

任务 3：KL 惩罚到底加在哪（约 60 分钟）

1. numpy 写一份按 token 的 KL，与 torch 对拍，**float64**：

```
# week09/day1/kl_toy.py — 两个词表分布上的 KL: numpy 手写 vs torch 对拍
import numpy as np, torch
rng = np.random.default_rng(0)                          # seed 写死，断言必须可复现
V = 50
def lsm(a): return a - np.log(np.exp(a).sum())          # log-softmax, float64
lp, lq = lsm(rng.normal(-3, 1, V)), lsm(rng.normal(-3, 1, V))
p = np.exp(lp)
t = lambda a: torch.tensor(a, dtype=torch.float64)
kl_np    = float((p * (lp - lq)).sum())
kl_torch = float(torch.nn.functional.kl_div(t(lq), t(lp), log_target=True, reduction="sum"))
print("kl_np =", kl_np, " kl_torch =", kl_torch)
assert abs(kl_np - kl_torch) < 1e-10, "KL 对拍失败: 检查 kl_div 的参数顺序"
print("KL CROSSCHECK OK")
```

2. 回答三个问题（各 ≥3 行，写进 NOTES.md）：① `KL(π‖π_ref)` 与 `KL(π_ref‖π)` 哪个惩罚「模型输出偏离 ref 里的高概率区」，为什么 RLHF 用前者；② KL 加进 reward（InstructGPT 风格，逐步奖励里减 β·KL）与加进 loss（PPO 目标里减）**等价到什么程度**；③ KL 系数 β 调大调小分别看到什么现象（**要能说出「β 太大 → 几乎不动、reward 平；β 太小 → reward 冲但输出变怪/长度失控」**）。

完成判据：打印 `KL CROSSCHECK OK`（差 <1e-10）；三问答案齐全且第 ③ 问给出两个方向各自的可观察现象（不是「会变好/变坏」）。

> 注：`F.kl_div(input, target)` 算的是 `KL(target‖input)`——**input 收的是对数概率、参数顺序和数学书写正好相反**，上面那句 `kl_div(lq, lp)` 才是 `KL(p‖q)`。你写 RL 相关代码时 90% 的「KL 值怪得没道理」都出在这里。今天先被它咬一口，比周 12 排查线上 A/B 数据时咬强。

任务 4：GAE 只到能白板讲清 + 收工（约 120 分钟）

1. 抄下递推式并标出每个符号：

```
δ_t = r_t + γ · V(s_{t+1}) − V(s_t)          # TD 误差
A_t = δ_t + γλ · A_{t+1}                      # 反向递推, A_T = 0
G_t^GAE(γ,λ) = Σ_{l≥0} (γλ)^l · δ_{t+l}
λ=0 → A_t = δ_t（单步 TD: 偏差大、方差小, 完全依赖 V 准不准）
λ=1 → A_t = Σ γ^l r_{t+l} − V(s_t)（Monte-Carlo 回报: 偏差小、方差爆炸）
```

2. 写一段 numpy 玩具实现，在一条 8 步轨迹上跑 λ=0 / 0.5 / 0.95 / 1.0，把四组 A_0 打出来：

```
# week09/day1/gae_toy.py
import numpy as np
r = np.array([0., 0., 0., 0., 0., 0., 0., 1.])            # 只有末步有奖励
V = np.array([.4, .35, .3, .25, .2, .15, .1, 0.])         # 一个不太准的 critic
def gae(r, V, gamma=1.0, lam=0.95):
    A = np.zeros_like(r); acc = 0.0
    for t in reversed(range(len(r))):
        nxt = V[t+1] if t + 1 < len(V) else 0.0
        acc = (r[t] + gamma*nxt - V[t]) + gamma*lam*acc      # 递推就这一行
        A[t] = acc
    return A
for lam in (0.0, 0.5, 0.95, 1.0):
    print(f"lam={lam:4.2f}  A_0={gae(r, V, lam=lam)[0]:+.4f}")
```

3. 一句话回答：**GRPO 为什么可以不要 Critic?** （同一 prompt 采 G 条，组内均值就是这条 prompt 的价值估计 `V(x)`，天然免费——代价是要多采样。）
4. NOTES.md 三行收工：五角色里你认为最容易被追倒的一个 + 今天没懂的 1 个点 + 明天推导 GRPO 从哪一行接上。

完成判据：脚本跑通打印 4 行；NOTES.md 记 λ=0 与 λ=1 两个 A_0 谁大谁小、以及一句「为什么 λ=1 时 Critic 准不准无所谓」；「GRPO 不需要 Critic」那一句能闭卷复述。

今日验收：rl-map 表填满 + 五角色表无空格 + `KL CROSSCHECK OK` + GAE 四行输出 + NOTES 新增 ≥6 条。

Day 2 • 周二：GRPO 组内基线推导 + RLVR

今日总览（6h）：今天把 GRPO 从「我知道它去掉 critic」推到「我能写出它的目标函数、说清每一项为什么长这样」。5 步推导、7 维对比表、闭卷默写，三件都要过硬。下午把 DeepSeekMath（GRPO 出处）与 DeepSeek-R1 读薄，写 2 条 200 字笔记——**Phase 2 论文配额在这里收尾**。

任务 1：GRPO 五步推导（约 120 分钟）

写进 `week09/day2/grpo-derivation.md`：

1. 起点仍是 KL 约束的策略优化目标 `max E[r] − β·KL(π‖π_ref)`，GRPO 不改变这个目标，**只改变优势怎么估**。
2. 对每个 prompt x，用旧策略采 G 条输出 `{y_1,…,y_G}`（**同组同 prompt，这是全部技巧的前提**）。
3. 组内基线：把每条的奖励做标准化 `Â_i = (r_i − mean(r_1…r_G)) / std(r_1…r_G)`。逐条说清：
   - 减 mean = 用组内均值充当 `V(x)`（替代 Critic），**为什么合理**：同一 prompt 下 G 条输出的期望奖励就是该 prompt 的价值。
   - 除 std = **尺度归一**，作用有二：不同 prompt 的奖励方差天差地别（简单题 G 条全对 → std=0；难题 → std 大），不归一的话难题的梯度会盖过简单题；同时它让 β/KL 系数不随任务难度漂移。**这一条正是上周「held-out 与训练集 accuracy 差异」的另一面：一组全对时优势恒为 0，模型从这条样本学不到任何东西。**
   - std=0（全对或全错）时优势为 0/未定义 → **实现上必须跳过这类组**，否则除零或引入无意义噪声。这是 Day 5 你要盯的第一个现象。

4. 逐 token 的策略更新项：对第 i 条输出的每个 token 位置 t，用概率比 `π_θ(y_i,t|x,y_i,<t) / π_θ_old(...)` 乘组内优势 `Â_i`，同样带 **clip**；GRPO 还把 KL **按 token 平均**（对照上周 DPO 的整句求和，这就是长度偏置差别的来源）。
5. 完整目标（照 DeepSeekMath 的形式写，标注分母 `G·|y_i|` 的含义）：

```
J_GRPO = E_{x,{y_i}} [ 1/(G·|y_i|) Σ_i Σ_t ( min(ρ_{i,t}Â_i, clip(ρ_{i,t},1±ε)Â_i) − β·D_KL[π_θ‖π_ref] ) ]
ρ_{i,t} = π_θ(y_{i,t} | x, y_{i,<t}) / π_θ_old(y_{i,t} | x, y_{i,<t})
```

完成判据：推导文档 5 步齐全；**能闭卷讲清「为什么除 std」的两条理由**（尺度/难度 + KL 系数不漂移），讲不出第 2 条就重来；NOTES.md 抄上完整目标式并标出 `Â_i` 是组内共享、不是 token 级。

> 注：`Â_i` 对同一条输出的**所有 token 是同一个数**（句级优势、token 级梯度），而 PPO 的 `Â_t` 是 token 级。这个区别决定了一个现象：GRPO 里一条答错的输出，它的每个 token 都被同等程度压低概率——包括中间写对的那几步。面试官问「GRPO 的信用分配有什么问题」，答案就在这里（也是 Math-Shepherd / 过程奖励想解决的事）。

任务 2：GRPO vs PPO 七维对比表（约 60 分钟）

默写这张表（`week09/day2/grpo-vs-ppo.md`），**这是周 14 八股的成品，不许留空格**：

维度	PPO	GRPO
优势估计来源	Critic + GAE	组内均值/标准差
在线模型份数	4（policy/value/ref/reward）+ 采样引擎	policy/ref + 验证器 + 采样引擎
显存	高（多份梯度与优化器态）	中（省掉 value 的一整份）
采样成本	每 prompt 1 条	每 prompt G 条（**这是省显存的代价**）
需要训练 RM 吗	是	否（奖励来自规则或外部验证器）
训练稳定性	较脆、超参敏感	较稳，但依赖奖励可验证性
失败模式	critic 估不准 → 优势噪声；KL 失衡 → 语言崩坏	组内全对/全错 → 无梯度；规则可钻 → 长度/格式 hacking

完成判据：7 行 × 3 列无空格，且「采样成本」一行写明 G 的量级（本周用 4-8）；口头 60 秒讲完「GRPO 用什么换了什么」（**答案要具体：用 G 倍采样换掉一整个 Critic 网络**）。

任务 3：RLVR 与规则奖励（约 60 分钟）

1. 写清 RLVR（RL with Verifiable Rewards）的一句话定义：奖励不来自人的偏好或 RM 的猜测，而来自**可机械判定的正确性**（答案数值对不对、括号匹不匹配、单元测试过不过、分类标签对不对）。
2. 列一张「可验证任务 / 验证器怎么写 / 能被怎么钻」的三行表（数学答案、代码测试、结构化 JSON 输出），每行必须给一个具体的钻空子方式。
3. 读 R1 报告里「规则奖励 + 长链思维」那节的要点，写 3 行：为什么 R1 选规则奖励而不是 RM（**答案要涉及 RM 在长链思维上不可靠 / 会被长度骗**）。

完成判据：三行表齐全且「钻空子」列每条具体到动作（不许写「模型会作弊」）；R1 笔记 ≥3 行且含「长输出更容易骗过打分器」这一含义。

> 注：本周你要写的三个验证器（GSM8K final-answer、格式合法、CEFR 分类）就是这张表的具体实现。**验证器写歪的后果比超参调错严重得多**：它每天给模型发几十万条「奖励」，一个提取漏洞 = 系统性地奖励错误答案。所以 Day 3 的核心判据是**人工核对 20 条**，不是脚本能跑。

任务 4：闭卷默写（约 60 分钟）

合掉一切，在 `week09/day2/recall.md` 闭卷写：

1. GRPO 的 `Â_i` 公式 + 「除 std 的两条理由」。
2. GRPO 完整目标函数（含 ρ、clip、KL 项、分母 `G·|y_i|`），并圈出与 PPO 唯一的不同来源。
3. 「GRPO 与 DPO 的差异：需不需要在线采样 / 需不需要 ref / 奖励是什么」。

完成判据：三题全写；`Â_i` 与目标函数**符号级正确**（允许 ε 写成 clip_range 这类等价记号，**不允许漏 KL 项或漏分母的 |y_i|**）；第 3 题三列答对。不合格当晚重默一遍，明天早上再默一次。

任务 5：论文笔记与收工（约 60 分钟）

DeepSeekMath（GRPO 出处）+ DeepSeek-R1，各 200 字写进 `notes/papers.md`。**只记「它为什么这样设计」**，不抄方法流程。再在 NOTES 写三行：今天最卡的一步 / 明天要写的三个验证器分别对应哪条判据 / verl 的心理准备（限时 110 分钟）。

完成判据：`notes/papers.md` 本周新增 2 条各 ≥180 字，每条至少一句以「它这么做是因为」开头；NOTES 三行齐全。

今日验收：grpo-derivation 五步 + 七维表无空格 + RLVR 三行表 + 闭卷默写判为过 + 论文 2 篇笔记。

Day 3 • 周三：规则奖励验证器三件套

今日总览（6h）：这是本周唯一「完全在你掌控中」的一天，也是 RLVR 的实操核心。三个验证器：**GSM8K 的 final-answer 提取 + 数值比对**、**输出格式合法校验**、**CEFR 难度分类正确性**（第 7 周教学数据的标签，为周 11 移植铺路）。**每个都必须先有单测再有实现**，并且人工核对 20 条提取结果。

任务 1：GSM8K 子集准备（约 70 分钟）

```
# week09/day3/prep_gsm8k.py — train 400 / test 200, seed 写死
import json, random, re
from datasets import load_dataset
random.seed(42)
ds = load_dataset("openai/gsm8k", "main")           # 打不开就改本地: load_dataset("json", data_files=...)
tr = ds["train"].select(random.sample(range(len(ds["train"])), 400))
te = ds["test"].select(random.sample(range(len(ds["test"])), 200))
def norm(ex):                                        # gold 归一化必须与 extract.canon 同构, 见注
    gold = re.sub(r"[^0-9+\-.]", "", ex["answer"].split("####")[-1])
    return {"problem": ex["question"], "answer": gold, "gold_full": ex["answer"]}
for name, d in [("gsm400", tr), ("gsm200", te)]:
    rows = [norm(x) for x in d]
    with open(f"E:/study/posttrain/week09/day3/{name}.jsonl", "w", encoding="utf-8") as f:
        for r in rows: f.write(json.dumps(r, ensure_ascii=False) + "\n")
    bad = sum(1 for r in rows if not r["answer"])    # 转换失败计数, 必须为 0
    print(name, len(rows), "| 无法转数值的条数:", bad, "| 示例:", rows[0]["answer"])
```

完成判据：两个 jsonl 分别 400 / 200 行；打印的「无法转数值的条数」=0；抽 5 条人工看 `gold_full` 与 `answer` 对应无误并记录这 5 条。

> 注：GSM8K 的 `answer` 末尾形如 `#### 42`，中间含逗号与 `$` 号；**gold 端与预测端两套归一化必须严格同构**（都走 `re.sub` 只留数字与正负号、小数点），否则会出现「gold 是 `1,200`、预测是 `1200`，明明对却判错」。这类不一致是 RLVR 里最隐蔽的一类假阴性：模型看起来「不收敛」，其实是奖励在乱发。**原始 `answer` 字段别就地改动，`gold_full` 保留原文，只在做比对键时归一化。**

任务 2：final-answer 提取器 + 20 条人工核对（约 100 分钟）

```
# week09/day3/extract.py — 提取模型输出里的最终答案
import re
ANS_TAG = re.compile(r"<answer>\s*([+-]?[\d,]*\.?\d+)\s*</answer>", re.I | re.S)
TAIL    = re.compile(r"(?:最终答案|the answer is|####)\s*[: :]?\s*([+-]?[\d,]*\.?\d+)", re.I)
LASTNUM = re.compile(r"([+-]?[\d,]*\.?\d+)(?!.*[0-9])", re.S)      # 兜底: 全文最后一个数字

def canon(s):
    return str(float(s.replace(",", "").strip())) if s else ""     # '1,200' 与 '1200' 归一到同一个键

def extract_final_answer(text):
    """优先级: <answer> 标签 > 「最终答案:」行 > 全文最后一个数字。提取不到返回 ''。"""
    for rx in (ANS_TAG, TAIL):
        m = rx.search(text)
        if m: return canon(m.group(1))
    m = LASTNUM.search(text)
    return canon(m.group(1)) if m else ""
```

单测（`week09/day3/test_extract.py`）：

```
from extract import extract_final_answer as ex
cases = [
    ("先算 3+2=5。\n<answer> 12 </answer>", "12.0"),
    ("总共花了 1,200 元。最终答案：1200", "1200.0"),        # 全角冒号: TAIL 里 [: :] 两个都不能少
    ("She has 3 apples. The answer is 3.", "3.0"),
    ("中间值 7 和 9，最后是 42", "42.0"),                    # 走 LASTNUM 兜底
    ("我不知道该答什么", ""),                                 # 提取不到必须是 ''
    ("<answer>abc</answer>然后是 8", "8.0"),                  # 标签内非数字 → 退化到最后一个数字
]
for txt, want in cases:
    got = ex(txt); assert got == want, f"{txt[:20]!r} -> got {got!r} want {want!r}"
print("EXTRACT TESTS PASS", len(cases))
```

完成判据：打印 `EXTRACT TESTS PASS 6`；**再从 `gsm400` 里让基座模型（无 GRPO）生成 20 条真实输出，逐条人工核对提取结果对不对，对 20/20 才算过**；任何一条判错的，写清是「提取错」还是「模型没写答案」，两类各计数。

> 注：那个「全文最后一个数字」的兜底是**双刃剑**，必须在 NOTES 里写一句「它会把解题过程中的中间数当答案」。面试问「你的奖励函数有什么已知漏洞」，你主动说出这一条，比被追问出来强十倍。更稳的做法是**格式奖励与正确性奖励分开发**：格式不合规直接给 0，不进提取环节——这正是任务 3 与任务 5 要做的事。

任务 3：格式合法校验器（约 60 分钟）

要求模型输出必须满足：含且只含一对 `<answer>…</answer>`、标签内是纯数字、`<think>` 段存在且非空。

```
# week09/day3/reward_format.py — 四档: 无推理段 0.0 / 标签数≠1 0.25 / 标签内有垃圾 0.5 / 全合规 1.0
import re
ONE   = re.compile(r"<answer>(.*?)</answer>", re.I | re.S)
THINK = re.compile(r"<think>(.*?)</think>", re.I | re.S)
def reward_format(text):
    tags = ONE.findall(text); th = THINK.search(text)
    if not th or len(th.group(1).strip()) < 8: return 0.0        # 没有推理段
    if len(tags) != 1: return 0.25                                # 0 个或多个 answer 标签
    if not re.fullmatch(r"[+-]?\d*\.?\d+", tags[0].strip().replace(",", "")): return 0.5
    return 1.0
```

完成判据：为四档各写一个用例并全部断言通过（**必须包含「两个 `<answer>` 标签」与「标签内 `12` 元」两条**）；函数返回值集合 ⊆ {0, 0.25, 0.5, 1.0}；四档判定逻辑作为注释贴在函数上方。

任务 4：CEFR 难度分类验证器（约 40 分钟）

复用第 7 周教学数据里的 CEFR 标签（A1/A2/B1/B2/C1），做**可机械判定**的分类正确性奖励，同时要求输出是合法 JSON——这两个条件就是周 11 移植到 tutor 的奖励雏形。

```
# week09/day3/reward_cefr.py — 奖励 = 格式 × 分类: 0.0 / 0.1(档位合法但判错) / 0.25(缺 key) / 1.0(对)
import json
LEVELS = ["A1", "A2", "B1", "B2", "C1"]
def reward_cefr(text, gold):
    try:
        obj = json.loads(text[text.index("{"): text.rindex("}") + 1])
    except (ValueError, IndexError):
        return 0.0
    if not isinstance(obj, dict) or "cefr" not in obj: return 0.25
    p = str(obj["cefr"]).strip().upper()
    if p == str(gold).strip().upper(): return 1.0
    return 0.1 if p in LEVELS else 0.0
```

完成判据：6 个断言全过（正确 / 判错但档位合法 / 非法档位 / 缺 key / 无 JSON / 多余文字包裹的 JSON），打印 `CEFR REWARD TESTS PASS 6`；`reward_cefr` 返回值全部落在 {0, 0.1, 0.25, 1.0}。

> 注：为什么给「判错但档位合法」0.1 而不是 0：它让模型先学会**输出一个合法档位**，再学判准。RL 里这叫形状先于质量的奖励塑形（shaping）。**风险也要写进 NOTES**：塑形给得太密，模型会满足于 0.1 而不去争 1.0（**局部最优是 RLVR 的第二种失败模式，比不收敛更常见**）。今天先埋这条观察，Day 5/6 若看到 reward 卡在低位不动，第一个查的就是它。

任务 5：合成奖励 + 退化用例 + 收工（约 90 分钟）

```
# week09/day3/reward.py — 合成奖励: 格式不合规直接归零, 否则 0.8×正确 + 0.2×CEFR
from extract import extract_final_answer, canon
from reward_format import reward_format
from reward_cefr import reward_cefr
def total_reward(text, gold_answer, gold_cefr="B1"):
    f = reward_format(text)
    if f < 1.0: return 0.0, {"fmt": f, "acc": 0.0, "cefr": 0.0}
    acc = 1.0 if canon(extract_final_answer(text)) == canon(str(gold_answer)) else 0.0
    c = reward_cefr(text, gold_cefr) if "cefr" in text.lower() else 0.0
    return 0.8*acc + 0.2*c, {"fmt": f, "acc": acc, "cefr": c}
```

两个必须存在的退化用例（写进 `test_reward.py`）：全对 8 条与全错 8 条——组内 std=0 时优势必须为 0/被跳过，这正是 Day 2 任务 1 第 3 步的实现验证。

```
import numpy as np
def group_adv(rewards):
    r = np.asarray(rewards, dtype=np.float64); s = r.std()
    if s < 1e-8: return np.zeros_like(r), True       # 跳过该组, 不许有 0/0
    return (r - r.mean()) / s, False
print(group_adv([1.0]*8), group_adv([0.0, 0.0, 0.8, 1.0, 0.2, 0.0, 1.0, 0.4]))
```

完成判据：`test_reward.py` 打印两个退化用例结果（第一组跳过=True、优势全 0；第二组跳过=False、优势有正有负）；`total_reward` 对一条真实 GSM8K 正确输出返回 ≥0.8、对一条错误输出返回 ≤0.2（**打印这两条的实际数字**，不许只写「测试通过」）；NOTES.md 写三行（今天最卡的点 / 明天 110 分钟限时的计时方式 / 三个验证器分别的已知漏洞）。

> 注：今天是本周唯一「零 GPU、全靠你自己」的一天，**做完它你就已经具备 RLVR 岗位的实际日常能力**（写验证器比调 verl 更常出现在工作里）。时间不够就优先保任务 2 的人工核对与任务 5 的退化用例，任务 4 可以只做函数不做单测——**但 20 条人工核对绝不许砍**，它是明天 reward 曲线能不能被解释的前提。

今日验收：gsm400/gsm200 行数达标 + `EXTRACT TESTS PASS 6` + 20 条人工核对 20/20 + 格式四档单测 + CEFR 六条单测 + 退化用例两组结论正确。

Day 4 • 周四：verl 环境搭建（本周最大风险）+ 降级路线

今日总览（7h）：今天的目标只有一个：**晚上睡觉前，手上有一条「能跑一步 GRPO 并打印出 reward」的路径**。首选 verl（业界主流、多家模型公司 JD 直接点名），**限时 110 分钟**；到点没通就**立刻**切 TRL `GRPOTrainer` + 0.5B + 小样本，不硬扛。**降级不是失败，按计划降级是执行力的证明**——这句话明天会写进博客。

任务 1：开卡算账 + 实例版本记录（约 50 分钟）

1. AutoDL 开 4090 24G（镜像 pytorch2.x + cuda12.1 + py3.10），开机第一件事把版本记死：

```
source /etc/network_turbo
mkdir -p /root/autodl-tmp/week09 && cd /root/autodl-tmp/week09
nvidia-smi | head -12 > instance-versions.txt
python -c "import torch; print('torch', torch.__version__, 'cuda', torch.version.cuda)" >> instance-versions.txt
pip list | grep -Ei '^(vllm|transformers|trl|flash-attn|xformers|deepspeed)' >> instance-versions.txt
cat instance-versions.txt                                    # 当天回传本地 week09/day4/
```

2. NOTES 写今日卡时预算：**Day 4 上限 3h**（¥5-8）。

完成判据：`instance-versions.txt` 有内容且回传到本地 `week09/day4/`；卡时预算写在 NOTES。**没有这份版本记录，明天任何报错你都无法定位是版本还是代码。**

> 注：verl 的安装文档要求 **vLLM / torch / CUDA 三者版本互相对齐**，这是它最容易翻车的地方。**别 `pip install vllm` 装最新版**，先按 verl README 的版本表选；装 verl 时用 `pip install verl --no-deps` 或源码 `pip install -e .`，防止它把你镜像里配好的 torch 悄悄换掉。这份警觉来自一个基本事实：**镜像里的 torch 是配好 CUDA 的，任何覆盖它的 pip 操作都可能让你从「能跑」变「CUDA 不可用」**。

任务 2：verl 安装与冲突排查（约 110 分钟，硬性限时）

1. 按仓库 README 走（**以 README 为准，不要照我这几行硬敲**，我给的只是动作顺序）：

```
cd /root/autodl-tmp/week09
git clone https://github.com/volcengine/verl.git && cd verl
python -c "import verl" 2>/dev/null || pip install -e . --no-deps
pip show verl | head -4                      # 没有版本信息 = 没装上
ls verl/trainer/config | head                # 找 grpo 相关那份示例配置
python -m verl.trainer.main_ppo --help 2>&1 | head -20    # CLI 起得来，说明通路是活的
```

2. **限时纪律**：110 分钟一到，无论到哪一步，**必须写一份 8 行的「现状说明」**（报什么错、试过什么、怀疑哪一层：依赖 / 显存 / 配置格式 / 数据格式），然后进任务 4 的降级路线。

完成判据：二选一即算完成本任务：① `pip show verl` 有版本 **且** `--help` 能打出参数说明（通路活着）；② 写了那份 8 行现状说明并转入降级路线。**「还在装」不算完成。**

> 注：verl 默认走多卡 + FSDP + vLLM 的分布式配置，**单张 4090 上你要改的东西不少**（`trainer.n_gpus_per_node=1`、rollout 的 `tensor_model_parallel_size=1`、vLLM 显存比例调低）。**如果 110 分钟里你一直在改配置而不是在读错误，说明你在猜**——停下来去 `examples/` 找有没有 grpo 的现成脚本，抄那个，别自己拼 hydra 配置。

任务 3：vLLM rollout 冒烟（约 60 分钟）

不管 verl 通没通，**rollout 引擎必须单独冒烟一次**，因为明天 G=4/8 采样的显存全靠它：

```
# week09/day4/rollout_smoke.py — 一次采 8 条，看显存与耗时
import time, torch
from vllm import LLM, SamplingParams
llm = LLM(model="/root/autodl-tmp/qwen05b", max_model_len=1536, dtype="bfloat16",
          gpu_memory_utilization=0.6, seed=42)
sp = SamplingParams(n=8, temperature=1.0, top_p=1.0, max_tokens=512, seed=42)
t0 = time.time()
out = llm.generate(["下面每题先把推理放在 <think> 与 </think> 之间，再把最终数字放在 <answer> 与 </answer> 之间: 2+3×4 = ?"], sp)
print("8 条耗时 %.1fs, 条数=%d, 显存峰值 %.1f GB" % (time.time()-t0, len(out[0].outputs),
      torch.cuda.max_memory_allocated()/2**30))
print("互不相同的条数 =", len({o.text.strip() for o in out[0].outputs}))   # 必须是 8
```

完成判据：脚本跑完打印耗时、峰值显存、**互不相同的条数 = 8** 三个数（temperature=1.0 下若 8 条一模一样，说明采样没打开，**明天组内优势会恒为 0**，这是必须先发现的致命问题）；`gpu_memory_utilization=0.6` 时 `nvidia-smi` 给训练侧留 ≥6GB。

> 注：GRPO 靠「同组 G 条不同」吃饭。采样温度、`n`、以及**是否复用同一条**这三件事任何一个错了，都会让 reward 曲线变成一条直线，而 loss 看起来仍在「正常波动」。**今天这条判据（8 条不同）就是明天所有诊断的第一块地基。**

任务 4：降级路线 —— TRL `GRPOTrainer` 冒烟（约 130 分钟）

用 0.5B + 64 题 + G=4 在实例上跑通一步 GRPO。**这段代码是本周最重要的兜底，必须在今天完成，不许推到明天。**

```
# week09/day4/grpo_trl_smoke.py
import json, re
from datasets import Dataset
from transformers import AutoModelForCausalLM, AutoTokenizer
from trl import GRPOTrainer, GRPOConfig
from extract import extract_final_answer, canon          # 昨天写好的验证器，直接复用

MODEL = "/root/autodl-tmp/qwen05b"
tok = AutoTokenizer.from_pretrained(MODEL)
rows = [json.loads(l) for l in open("gsm400.jsonl", encoding="utf-8")][:64]
def make_ex(r):
    msg = [{"role": "user", "content": "先把推理放在 <think> 与 </think> 之间，再把最终数字放在 <answer> 与
</answer> 之间。题目: " + r["problem"]}]
    return {"prompt": msg, "answer": r["answer"]}
ds = Dataset.from_list([make_ex(r) for r in rows])
text_of = lambda cs: [c[0]["content"] if isinstance(c, list) else c for c in cs]   # 版本差异兜底，见注
def reward_acc(completions, answer=None, **kw):
    got = [canon(extract_final_answer(o)) for o in text_of(completions)]
    return [1.0 if g and g == canon(str(a)) else 0.0 for g, a in zip(got, answer or [])]
def reward_fmt(completions, **kw):
    return [1.0 if re.search(r"<answer>\s*[+-]?[\d,]*\.?\d+\s*</answer>", o, re.I|re.S) else 0.0
            for o in text_of(completions)]

model = AutoModelForCausalLM.from_pretrained(MODEL, torch_dtype=torch.bfloat16)
cfg = GRPOConfig(output_dir="/root/autodl-tmp/out/grpo-smoke",
                 per_device_train_batch_size=4, gradient_accumulation_steps=1,  # 必须能被 num_generations
整除
                 num_generations=4, max_completion_length=256, max_prompt_length=512,
                 beta=0.04, temperature=1.0, top_p=1.0,   # beta=0.04 只是常见起点量级，见注
                 learning_rate=1e-6, num_train_epochs=1.0, bf16=True,
                 gradient_checkpointing=True, logging_steps=1, report_to="wandb",
                 seed=42, data_seed=42)
trainer = GRPOTrainer(model=model, reward_funcs=[reward_fmt, reward_acc],
                      args=cfg, train_dataset=ds, processing_class=tok)
trainer.train()
```

`python grpo_trl_smoke.py 2>&1 | tee smoke.log`

完成判据：至少跑出 3 个 log 步且每步同时打印出 **reward、KL（指标名含 `kl` 即可）、completion 长度** 三项；把「这些字段在你这个 trl 版本里到底叫什么」记进 NOTES。

> 注：**这个文件的参数名很可能与你装到的 trl 版本不完全一致**—— `GRPOTrainer` 的签名在 0.x 各版本之间改过几轮（`reward_funcs` 收到的 `completions` 是消息列表还是字符串、`max_prompt_length` 放在 config 还是 tokenizer、`num_generations` 的整除约束）。**正确做法不是上网抄另一版**，是打印签名自己看：`python -c "import inspect, trl; print(inspect.signature(trl.GRPOTrainer.__init__)); print(inspect.signature(trl.GRPOConfig.__init__))" | tee grpo-signature.txt`
>
> 注：`per_device_train_batch_size` × 卡数 必须能被 `num_generations` 整除，这是 GRPO 的硬性约束（**一个组不能被劈到两个设备上**）。违反时框架给的信息通常很难读，报错长相像 shape mismatch。**记住这条约束本身，比记住某次报错有用**——面试会问「GRPO 的 batch 为什么要整除」。

任务 5：验证器接进训练 + 指标落盘（约 70 分钟）

1. 用 `week09/day3/reward.py` 的 `total_reward` 替换上面的两个简易函数（它返回 `(value, dict)` 元组，接入时返回第一个元素，把 dict 另存进日志），确保 **格式不合归零** 这条硬规则在训练里真的生效（拿一条不合格式的输出手工验证一次）。
2. 确认三条曲线各自有名字：reward（或含 `reward` 的指标）、completion 长度、KL。连不上 wandb 就 `export WANDB_MODE=offline`（之后 `wandb sync <目录>` 补传），同时**逐行写进 jsonl**——明天你要靠它画图，不能只靠截图。

```
def log_step(step, d, path="grpo_metrics.jsonl"):      # 与 wandb 能否连上无关，独立可用
    open(path, "a", encoding="utf-8").write(json.dumps({"step": step, **d}) + "\n")
```

完成判据：一次 3-step 冒烟同时产出 ① wandb 截图或 offline 目录 ② `grpo_metrics.jsonl`（≥3 行、每行含 reward/长度/KL 三个键）；手工验证「格式不合规 → total_reward=0」的那条打印留档。两者都有才算通——**明天不收敛时，你只能靠这份 jsonl 做诊断**。

今日验收：instance-versions.txt 回传本地 + verl 状态为「活」或「8 行说明」二者之一 + rollout 8 条互不相同 + GRPO 冒烟 3 步出三条指标 + metrics jsonl 存在 + 已关机（**关机 ≠ 销毁**）。

> 注：今天如果 verl 没通、只有 TRL 路线通了，**本周主线照走不误**——简报与总计划都把降级路线写成正式方案。但 **8 行现状说明必须写清**：写到「哪个包哪个版本报什么错、我怀疑是 X、下一步该 Y」的质量。这段文字 Day 6 直接进博客、周 14 直接进失败案例清单，**它是你今天真正的第二产出**。

Day 5 • 周五：GSM8K 子集跑 GRPO，画 reward / 长度 / KL 三条曲线

今日总览（6h）：正式实验日。上午定采样与显存预算，开卡跑 400 题 GRPO（G=4 起，够用再上 8），全程记 reward / 输出长度 / KL 三条曲线；下午做健康判据检查与不收敛应急。**今天开始就要有「可能不收敛」的心理与文档准备，这是计划的一部分，不是意外。**

任务 1：采样配置与显存预算（约 50 分钟）

在 `week09/day5/run-config.md` 写死配置表，每项标理由：

项	值	理由
模型	Qwen2.5-0.5B（主）/ 1.5B（若 0.5B 一天跑完有余量）	4090 24G 上 rollout + 训练要留够空间
数据	gsm400.jsonl（seed 42）	昨日已准备，train 侧
G（num_generations）	4 起步	组内均值当基线的最少样本；G 太小 std 噪声大
max_completion_length	256（可升 384）	太长 → 长度 hacking 与显存双向风险

项	值	理由
lr	1e-6	RL 阶段比 DPO 再低一个量级的常见做法
beta（KL 系数）	0.04	起点，今天看 KL 曲线决定是否升到 0.08
步数	约 100 step 或 40 分钟墙钟，**取先到者**	卡时预算硬约束
seed	42（`seed` 与 `data_seed` 都写）	可复现

完成判据：配置表 8 行齐全且每行有「理由」（**空理由的那行不许用**）；卡时预算写在 NOTES（今日 ≤5h 4090，¥8-12）。

任务 2：正式跑 GRPO 并记录（约 150 分钟）

1. 上卡，把昨日冒烟脚本换成 `gsm400` 全量（启动脚本先写好再开卡）：

```
source /etc/network_turbo && cd /root/autodl-tmp/week09
python grpo_trl.py 2>&1 | tee grpo_main.log          # 后台跑用 nohup，但每 10 分钟必须回来看一眼
nvidia-smi --query-gpu=memory.used --format=csv -i 0   # 顺手记峰值
```

2. 每 20 step 记录四个数：`reward`、`reward_std`（组内 std）、`completion_length`、`kl`（落盘通路昨天已打通，今天只补一次显存峰值）。
3. 若中途 OOM：先把 `max_completion_length` 256→192、`per_device_train_batch_size` 降一档，**不要先动 G**（G 降了组内基线就变差，等于改算法）。

完成判据：跑完 ≥60 step（含 60 即算完成，不足 60 记为「采样通路未验证」）；`grpo_metrics.jsonl` ≥60 行；`grpo_main.log` 里能翻到一次显存峰值数字；**三条指标每一步都有值**（不是前几步有、后面 NaN）。

> 注：**NaN 是 GRPO 的头号事故**。最常见的三个来源：① 某组 std=0 你没跳过，优势里出现 0/0 ② logprob 累加用了 bf16 而不是 fp32 ③ lr 太大第一步就把策略打崩，KL 爆掉后 log-ratio 溢出。任何一步看到 NaN，**立刻停、记下步号与当时四个数**，再按这三条排查。这张「哪一步、哪四个数」的记录，就是 Day 6 博客里最值钱的那段诊断。

任务 3：三条曲线健康判据（约 70 分钟）

画图（本地 matplotlib，从 jsonl 读，不依赖 wandb）：

```
# week09/day5/plot3.py
import json, matplotlib
matplotlib.use("Agg"); import matplotlib.pyplot as plt
rows = [json.loads(l) for l in open("grpo_metrics.jsonl", encoding="utf-8")]
keys = [k for k in rows[0] if k != "step"]
print("可用指标:", keys)                                    # 先把字段名打出来，别凭想象猜
pick = {"reward": [k for k in keys if "reward" in k and "std" not in k][0],
        "len":    [k for k in keys if "len" in k][0],
        "kl":     [k for k in keys if "kl" in k][0]}
fig, ax = plt.subplots(3, 1, figsize=(8, 9), sharex=True)
for a, (name, k) in zip(ax, pick.items()):
    a.plot([r["step"] for r in rows], [float(r[k]) for r in rows], marker="o", ms=3)
    a.set_ylabel(name); a.grid(alpha=.3)
ax[-1].set_xlabel("step"); fig.tight_layout(); fig.savefig("week09-day5-curves.png", dpi=120)
```

判据表（逐条写「观察到什么 + 结论」）：

曲线	健康	不健康	你观察到的是哪一类
reward	上行（或前段平、后段起）	恒平（无信号）/ 冲高后暴跌（崩了）/ NaN	
completion 长度	平稳或缓慢变化	暴涨而 reward 平 → **reward hacking**	
KL	有界、量级 <1（0.5B 上常见 <0.5）	单调上冲不回头 → β 太小 / lr 太大	

完成判据：`week09-day5-curves.png` 生成；三条各有一句「观察到 + 结论 + 依据数字」的判读（**不许出现「大致正常」这种没有数字的说法**）；若某条无法判读（比如全是 0），写清它意味着哪一步失效。

任务 4：不收敛应急三板（约 50 分钟，需要才做）

按顺序试，**每试一项只跑 20 step 并记录四条指标**（不许同时改两项，那是最典型的无效 debug）：

1. **降 lr**（1e-6 → 5e-7）：治 KL 上冲 / loss 抖。
2. **升 β**（0.04 → 0.08）：治 reward 涨但输出变怪、KL 失控。
3. **升 G**（4 → 8）或**换更简单的题**（只留一步计算题）：治 reward 恒 0（模型一条都做不对，组内全是 0，没有正样本可学）。

完成判据：做了哪一板、改前改后各 20 step 的四个指标数字，写成一张小表进 `week09/day5/emergency.md`；**三板都试完仍不收敛也完全合格**，只要这张表在（它就是 M3 的诊断结论）。

> 注：第 3 板「reward 恒 0」的成因特别值得写清：**0.5B 模型在 GSM8K 上一步做对的概率可能非常低**，如果 400 题里绝大多数组的 4 条全错，你就在拿 0 梯度训练。这时**不是模型不行也不是代码不行，是任务与模型规模不匹配**——换成更小的子集（只用一步计算题）或把 `max_completion_length` 放宽让模型有时间推对。这个判断能力就是「会做 RL」和「会调 verl」的分界。

任务 5：关机与备份（约 40 分钟）

```
cd /root/autodl-tmp && zip -r backup-grpo.zip out/grpo-smoke week09/grpo_metrics.jsonl
cd /root/autodl-tmp/llm-posttraining-lab && git add m3-grpo && git commit -m "week9 grpo run" && git push
# 曲线 png 与 metrics jsonl 回传本地 E:/study/posttrain/week09/day5/，再去控制台确认「已关机」
```

完成判据：`grpo_metrics.jsonl`、`grpo_main.log`、`week09-day5-curves.png` 三件在本地各有一份；控制台状态「已关机」；NOTES 记下今日实际卡时与花费两个数字。

> 注：曲线图与 metrics 一定**当天回传本地**。实例上的数据盘随时可能因为误操作或平台回收没了，**关机 ≠ 备份**；丢了这天的曲线，Day 6 的博客就只能写感想。

今日验收：≥60 step + 三曲线 png + 三条判读带数字 + 应急表（若需要）+ 已关机与双备份。

Day 6 • 周六：准确率对照 + hacking 小实验 + 失败案例 + 博客

今日总览（7h）：把昨天的曲线变成结论和输出。上午跑 baseline 与训练后在 `gsm200` 上的准确率对照（**同题、同 seed、同 max_tokens 配置写清**）；中午做一次「只奖励长度」的小实验，正面制造 reward hacking；下午整理 3 条失败案例，晚上把第 3 篇博客发出去。**这一天的文档质量决定 M3 在面试里的价值。**

任务 1：baseline vs 训练后准确率（约 100 分钟）

```
# week09/day6/eval_acc.py — gsm200 上的 pass@1，两侧同配置
import json, re
from vllm import LLM, SamplingParams
from extract import extract_final_answer, canon
TEST = [json.loads(l) for l in open("gsm200.jsonl", encoding="utf-8")]
PROMPT = "先把推理放在 <think> 与 </think> 之间，再把最终数字放在 <answer> 与 </answer> 之间。题目："
sp = SamplingParams(n=1, temperature=0.0, top_p=1.0, max_tokens=384, seed=42)   # temp=0: 判据必须可复现
def acc(model_path, tag):
    llm = LLM(model=model_path, max_model_len=1024, dtype="bfloat16", gpu_memory_utilization=0.85,
seed=42)
    outs = llm.generate([PROMPT + r["problem"] for r in TEST], sp)
    hit = fmt = 0
    for o, r in zip(outs, TEST):
        t = o.outputs[0].text
        if re.search(r"<answer>.*</answer>", t, re.I|re.S): fmt += 1
        hit += canon(extract_final_answer(t)) == canon(str(r["answer"]))
    print(f"{tag}: n={len(TEST)} acc={hit}/{len(TEST)}={hit/len(TEST):.2%} 格式合规={fmt/len(TEST):.2%}")
acc("/root/autodl-tmp/qwen05b", "baseline(0.5B)")
acc("/root/autodl-tmp/out/grpo-main/merged", "GRPO 后")
```

完成判据：两个 `acc=…` 与两个 `格式合规=…` 共四个百分数打印出来并记入 `m3-grpo/results.md`，**两侧 `n=` 都必须是 200**；GRPO 后 acc ≥ baseline 记「正向」；**低于 baseline 也合格**，但必须给出 ≥3 行诊断（数据太少？长度截断？格式合规率下降？KL 过大？）——**这 3 行诊断比涨 5 个点更值钱**。

> 注：**temperature=0 做评测、temperature=1.0 做训练采样**，这两件事不能互换：评测要可复现的判据，训练组内要多样性。Day 4 你已经见过混用的后果（8 条一模一样 → 组内优势恒 0）。另外 `max_tokens=384` 与训练时的 `max_completion_length=256` 不同——**评测放宽长度是合理的，但必须写明**，否则「训练时答案被截断、评测时没被截断」会伪装成能力提升。

任务 2：「只奖励长度」的 hacking 小实验（约 80 分钟）

把奖励换成 `reward = 输出字符数 / 300`（截到 1.0），**不给正确性任何权重**，其余配置与昨天完全一致，只跑 30 step。

```
# week09/day6/reward_hacky.py — 故意只做坏事的一次实验
def reward_length_only(completions, **kw):
    outs = [c[0]["content"] if isinstance(c, list) else c for c in completions]
    return [min(1.0, len(o) / 300.0) for o in outs]
```

完成判据：30 step 内产出一条长度曲线与一条 reward 曲线；写 4 行结论，**必须含具体数字**（形如「平均长度从 X 涨到 Y、顶到 `max_completion_length` 的比例是 Z%，同期正确率从 A% 掉到 B%」）；长度曲线截图存 `m3-grpo/hacking.png`。

> 注：这条实验的产出**必然是 hacking**（模型会把长度推到截断上限），这不是失败，是设计如此——它的唯一目的是让你**亲手在曲线上认出 reward hacking 的形状**：reward 一路向上、真实质量一路向下、长度顶到天花板。**下次面试问「reward hacking 出现过吗」，你要能说出「我专门造了一次，它长这样，我用哪三个指标发现的」**。若卡时不够（动用 A100 就要砍这里），改成本地 0.5B 跑 10 step，只取「长度曲线」一条，结论照样成立。

任务 3：失败案例 3 条（约 70 分钟）

从本周所有真实卡点里挑 3 条，写成统一格式（`m3-grpo/failures.md`）：

```
### 案例 N：<一句话现象>
- 现象：（带步号/数字，例如「step 12 起 reward 恒为 0」）
- 定位过程：（看了哪三个指标、排除了什么、怀疑什么）
- 根因：（版本 / 配置 / 数据 / 算法理解，四选一，写具体）
- 修复或绕过：（改了什么、花了多久、有没有验证）
- 面试里我会怎么讲（2 行）
```

完成判据：3 条案例齐全，每条五个小节无空白；**至少一条的根因落在「对 GRPO 机制的理解」而不是环境配置**（否则面试官会认为你只会装包）；每条都有数字或明确观察，不许写「试了很多次终于好了」。

> 注：候选不用编：Day 4 的 verl 现状说明、Day 5 的 NaN/低信号记录、Day 3 那条「0.1 局部最优」的奖励塑形风险、以及今天 acc 下降的诊断，都是现成素材。**如果你本周一切顺利、一条失败都找不出来，那本身就是最值得写的一条**——多半说明你的判据不够严（比如 reward 恒 0 你没看出来），照这条写成「差点没发现的坑」，比硬编三条假失败诚实得多。

任务 4：README、周复盘与归档（约 80 分钟）

1. `m3-grpo/README.md` 六段：这是什么 / 结果表（acc 前后、三条曲线判读）/ 复现步骤（**含环境版本**）/ 失败案例 3 条摘要 / 已知局限（模型只有 0.5B、只跑 1 个 seed、G=4、没和 PPO 实测对比）/ 诚信声明（**哪些**

是复现 TRL/verl 官方示例、哪些是你自己写的：`extract.py`、三个 reward、`plot3.py`、`eval_acc.py` 是你写的）。

2. 三张曲线图、`grpo_metrics.jsonl`、两份配置 json 全部入 repo：`cd E:/study/posttrain/llm-posttraining-lab && git add m3-grpo && git commit -m "M3: GRPO on GSM8K subset" && git push`。
3. `week09/day6/review.md` 四块：硬产出清单 / 本周实际卡时与花费 / 最大的一个未知 / 周 10 第一件事。

完成判据：README 六段齐全，「已知局限」≥3 条（**不许写「效果可能不够好」这类空话，要写「只有 0.5B / 只有 1 seed / G=4 未做 8 对照」**）；`git push` 后 GitHub 能翻到图与 jsonl；诚信声明那段存在；review.md 四块齐全。

任务 5：第 3 篇博客成文并发布（约 90 分钟）

标题固定：**《GRPO 第一次实操：reward 曲线教会我的事》**（简报博客配额第 3 篇，勿另开新篇；专栏名《从全栈工程师到后训练》）。结构 1800-2800 字：

1. 我为什么跳过 PPO（**显存量级手算那张账，直接引用 Day 1，并标明是估算**）。
2. GRPO 到底省掉了什么、代价是什么（组内均值当基线 → 拿 G 倍采样换一整个 Critic）。
3. 三条曲线我怎么读（贴图，reward / 长度 / KL 各一段，**每段带真实数字**）。
4. 我专门造了一次 reward hacking（贴图，说清它长什么样）。
5. 哪些没做出来 / 失败了（**这一节必须有，不许只写成功**）。
6. 下一步：把它搬到我自己的真实产品上（M4 预告，一句带过，不展开）。

完成判据：博客已发布且链接可访问，链接贴进 NOTES.md 顶部；正文 ≥2 张图（三曲线 + hacking 长度曲线）；第 3、4 节各有 ≥1 个具体数字；第 5 节存在且不空。

> 注：博客里**不要写「GRPO 比 PPO 好」**。写「在我的单卡预算下 PPO 跑不动，GRPO 是我唯一能跑通的选择，代价是 G 倍采样」。前者是八股，后者是判断力——面试官只在后者上面花时间。

今日验收：四个百分数进表 + hacking 实验有曲线与数字 + 失败案例 3 条 + README 六段 + 博客已发布。

Day 7 • 周日：补漏 + 闭卷复测 + 总验收 + 下周预习

今日总览（4h）：不加实验。补两个洞，闭卷复测三道，做 7 项总验收，预习周 10 的 M4 立项。**今天最重要的动作是承认哪些洞要留给后面几周。**

任务 1：补漏（约 80 分钟）

打开 `week09/gaps.md`。**规则：最多补 2 个洞，贪多必失。** 优先补：

1. `extract_final_answer` 的兜底漏洞（把中间数当答案）——补法：加一条规则（**只在最后一个 `</think>` 之后取数**），重跑昨天那 20 条人工核对，记下改了哪几条判定。
2. GRPO 目标式里 KL 的按 token 平均为什么影响长度偏置——补法：重读上周 Day 2 任务 1 第 6 步第 ③ 点，写 5 行对照（DPO 整句求和 vs GRPO 逐 token 平均，各自的长度激励）。

其余原样交给后续周，今天不碰。

完成判据：两个洞各有一份产出（一份新单测结果 或 一段 ≥5 行的解释），gaps 里标 `[x]` 并写「怎么补的」。

任务 2：闭卷复测（约 45 分钟，别跳）

白纸闭卷三题：

1. 写 GRPO 的 $\hat{A}_i$，说明为什么减 mean 能替代 Critic、为什么还要除 std（**两条理由都要**）。
2. PPO 五角色表默写（含「谁存优化器态」列）+ 一句「Critic 与 RM 的区别」。
3. 「你的 reward 曲线前段什么形状？哪一步开始没信号？你改了什么？」——按本周真实数字答。

完成判据：三题全答；第 1、2 题符号/结构无误；第 3 题报得出**具体步号或指标值**（说不出数字 = 本周曲线你没亲自读过，回 Day 5 把 jsonl 重看一遍）。

> 注：第 3 题是 M3 面试最可能的开场（「讲讲你的一次 RL 实验」）。能报出「step 12 之后 reward 恒 0，因为 0.5B 在 GSM8K 上一条都做不对，组内 std=0」这句话的人，和说「训练了很久 loss 慢慢降」的人，不是一个档次。

任务 3：总验收（约 30 分钟）

#	验收项	判据
1	PPO 白板能力	五角色表无空格；`KL CROSSCHECK OK`；7B 显存量级手算在 NOTES（标明是估算）
2	GRPO 推导	五步推导齐全 + 七维对比表无空格 + 闭卷默写符号级正确
3	验证器三件套	提取/格式/CEFR 单测全绿；**20 条人工核对 20/20**；两组退化用例结论正确
4	环境	verl 状态为「活」或「8 行现状说明」之一；TRL `GRPOTrainer` 冒烟 3 步出三条指标（**降级路线必须通**）
5	实验	≥60 step；三曲线 png 生成；三条判读各有数字
6	结果对照	gsm200 前后 acc 与格式合规四个百分数入表；未涨也有 ≥3 行诊断
7	输出	`m3-grpo/README.md` 六段（含已知局限与诚信声明）+ 失败案例 3 条 + 博客已发布

≥6 项通过 = 本周合格 = M3 达成。特别注明：本表没有任何一项要求「reward 收敛」或「准确率提升」；第 5、6 项的判据是「曲线与诊断在不在」。这是总计划风险预案第 1 条的落地——**不收敛也算完成，失败的曲线与诊断过程本身就是面试素材**。不足 6 项，把缺的写进周 10 Day 1 的前 2 小时，**但 M4 的立项文档不许因此延期**。

完成判据：7 行逐行填「通过 / 未通过 + 证据文件路径」，无空白格；判「未通过」的行在 `week09/gaps.md` 有对应条目（写了归到哪一天、做什么），零未通过时该行写「无」。

任务 4：下周预习（约 30 分钟）

`notes/preview-week10.md` 写五条：

1. 打开 `llm-posttraining-lab/`，确认 `m4-capstone/` 在位（没有就建空目录，明天要写立项文档）。
2. 回读第 7 周数据集的 data card：条数、主题分布、CEFR 标签覆盖比例——**明天选数据要靠它**。
3. 明日记忆里 tutor 相关的现有链路（`tutor_use_llm` 式开关在哪、一次 API 调用大致多少 token）——**用你已有的工程知识，不查资料**。
4. 写下你希望 M4 的「成功」是什么（先凭直觉写 3 行，明天 Day 1 任务 2 会把它变成可验证门槛）。
5. 一句提醒：**周 10 起总计划允许投日常实习**，但正式投递与简历终版在周 13——本周什么都不做。

完成判据：五条各 ≥2 行落盘 `notes/preview-week10.md`（30 分钟写不完理想长度，宁可短也要五条齐全）；第 2 条的三个数字（条数 / 主题分布 / CEFR 覆盖率）直接引自 data card 原文（能指出是哪一行）；第 5 条存在且写的是「本周不投」。

任务 5：归档与账面收口（约 55 分钟）

1. 三条曲线图、`grpo_metrics.jsonl`、失败案例、8 行 verl 现状说明——四件各自确认在**本地与 GitHub 各有一份**（实例随时可能没）。
2. 账面：本周卡时上限 16h / 实际 **h、预算 ¥35 / 实际 ¥**、是否动用 A100（若是，写明砍了 Day 6 任务 2 的哪部分）、API ≤¥8 / 实际 ¥__。四对数字写进 `week09/day7/review.md`。
3. 把「下周 Day 1 前 2 小时要补的东西」（若有）单独置顶写进 week10 的 gaps 开头，**不要靠记忆**。

完成判据：四件产物双备份确认（本地路径与 repo 路径各四条）；四对账面数字齐全；下周 Day 1 的补课条目要么写明「无」，要么具体到「哪天哪两小时做什么」。

> 注：M3 是四个里程碑里唯一「结果可能是坏的」那个，所以它的产物**必须靠归档而不是靠记忆**。两周后你回忆「那次 GRPO 为什么不收敛」，能想起来的是这份 jsonl 与这 55 分钟，不是当天熬夜时的感觉。周 14 的失败案例清单就直接从这里复制。

今日验收：两个洞有产出 + 复测三题过 + 验收表 ≥6 项打勾 + 预习五条 + 双备份与账面四对齐全。

本周暴露的洞 → 后续哪一周补

洞	归到	那天的动作
规则奖励只覆盖 GSM8K 数学，教学场景的验证器还没真用过	**周 11 • RLVR 移植**	把 `reward_cefr` + 格式校验接到 tutor 数据上（本周已备好这两个函数，周 11 就是搬过去跑）
G=4 与 G=8 没做对照；std 归一对梯度尺度的影响只是推理	周 11 · 对齐阶段消融	有余量就同一份数据跑 G=4/8 两档，画两张 reward 图对照
没实测过 PPO，五角色只到能讲	周 14 · 八股与失败案例	明确写成「讲得清、未实操」，**简历不写 PPO 实操**
GRPO 只有 1 个 seed，方差未知	周 12 · 评测与集成	自建教学评测集时同样本重跑 3 次做一致性抽查，先建立「单次结果不可信」的纪律

洞	归到	那天的动作
过程奖励 / Math-Shepherd 只知道名字	周 12 后或社招补强	留在论文清单，不排进主线

三、常见问题与纪律

- **verl 装不上算不算本周失败？** 不算。降级路线（TRL `GRPOTrainer` + 0.5B）是简报与总计划里写明的**正式方案**。判据是「有一条能跑通 RL 循环的路径 + 一条 verl 现状说明」。**但如果 110 分钟限时你没用、硬扛到凌晨——那才是真正的失败**，因为它把 Day 5 的实验时间吃掉了。
- **不收敛怎么办？** 按 Day 5 任务 4 三板走一遍并留数字，然后继续走 Day 6，写进 README 的「已知局限」与博客第 5 节。**记录得干净的不收敛 > 来路不明的高分。**
- **卡住多久该求助？** 单点 40 分钟：写 NOTES → 搜报错原文 + 对应框架仓库 issue → 再 20 分钟无解就切降级路线并立 gaps 条目。**环境类问题（版本冲突）永远优先绕过而不是修好**，你的目标是跑出曲线。
- **钱**：卡时 ≤16h、目标 ≈¥35（4090 主力 11h + 3h 余量）；动用 A100 2h（¥6-12/时）必须同步砍 Day 6 任务 2 的实验规模；API ≤¥8；全周上限 ¥43。周日核对四对预算/实际数字（Day 7 任务 5）。
- **实验纪律**：一次只改一项；每步指标落 jsonl（不依赖 wandb 能不能连上）；seed 写死；评测 temperature=0、训练采样 temperature=1.0，两者不许互换；`beta=0.04` 这类数值写进文档时标「常见起点量级」，**不许写成自己实测的结论**。
- **面试诚信**：README 与博客都要写清「复现自官方 / 我自己写」。本周你自己写的东西不少（三个验证器、提取器、画图脚本、评测脚本），**这些才是 RLVR 岗位真正要看的**；`GRPOTrainer` 本身不是你写的，也不需要假装。
- **别提前动 M4**。今天写好的那五条预习，明天 Day 1 才动。